Các kĩ thuật phục hồi CSDL

Nội dung

- Lịch trình khả phục hồi.
- Tổng quan về phục hồi.
- Kĩ thuật Write-Ahead Logging.
- Kĩ thuật phục hồi dựa trên Deferred Update.
- Kĩ thuật phục hồi dựa trên Immediate Update.
- Kĩ thuật phục hồi dựa trên Shadow Paging.

LỊCH TRÌNH KHẢ PHỤC HỒI

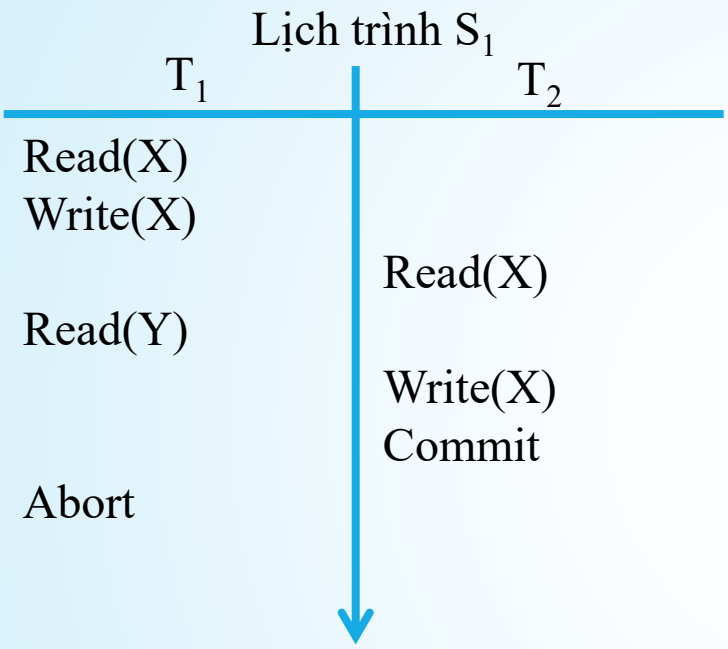
Tính khả phục hồi của lịch trình

- Trong việc tìm hiểu về điều khiển song hành, ta chưa xét nhiều đến sự thất bại của giao dịch.
- Nếu giao dịch Ti thất bại vì lý do nào đó (thường là các sự cố - failures), ta cần hủy bỏ giao dịch này để đảm bảo tính nguyên tử của giao dịch.
- Và để đảm bảo tính nhất quán, ta cần phải hủy bỏ tất cả các hiệu quả liên quan của giao dịch T.

Tính khả phục hồi của lịch trình

- Một số lịch trình dễ dàng phục hồi trong khi 1 số khác không thể phục hồi.
- Lịch trình mà có các giao dịch sau khi đã được bàn giao (Commit) không bao giờ phải rollback lại gọi là lịch trình khả phục hồi.
- Với mỗi cặp giao dịch Ti và Tj trong lịch trình khả phục hồi: nếu Ti đọc hạng mục dữ liệu được ghi bởi Tj thì lệnh commit của Tj phải diễn ra trước lệnh commit của Ti.

Ví dụ



- Giả sử trường hợp T1 gặp sự cố và phải rollback.
- T2?
- ➔ Lịch trình không thể phục hồi và không được phép.

Ví dụ

Lịch trình S_2

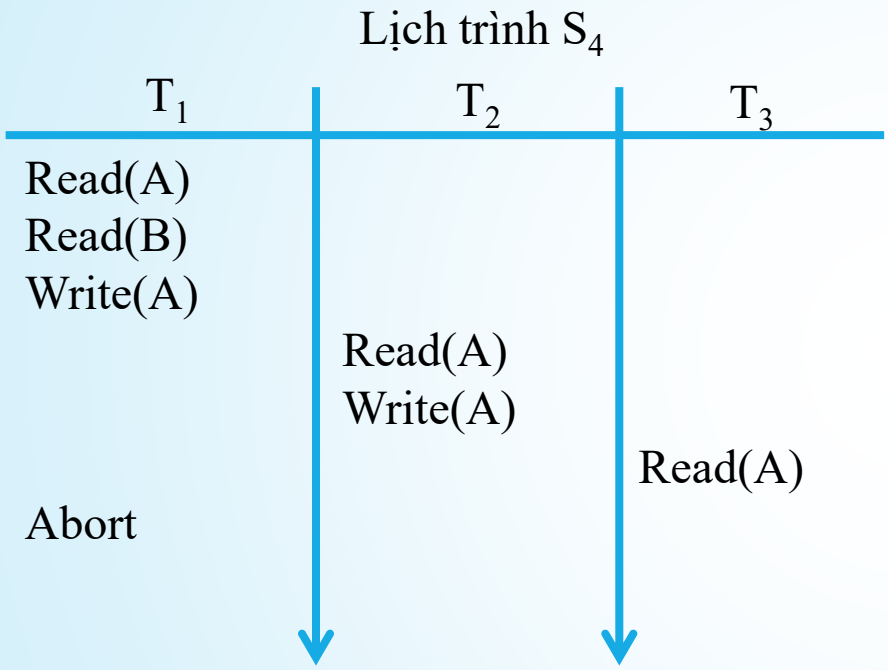
T_1	T_2
Read(X)	
	Read(X)
Write(X)	
Read(Y)	
	Write(X)
	Commit
Write(Y)	
Commit	

Lịch trình S_3

T_1	T_2
Read(X)	
Write(X)	
	Read(X)
Read(Y)	
	Write(X)
Write(Y)	
Commit	
	Commit

Khả phục hồi?

Lịch trình Cascadeless



- Ngay cả khi lịch trình là khả phục hồi, việc phục hồi đúng sau thất bại của một giao dịch cũng xảy ra vấn đề.
- Việc rollback của S4 diễn ra như thế nào?

Lịch trình Cascadeless

- Hiện tượng 1 giao dịch thất bại kéo theo một loạt các giao dịch khác phải rollback gọi là sự cuộn lại hàng loạt (cascading rollback).
- Việc này dẫn đến việc hủy bỏ một khối lượng công việc đáng kể.

- Các lịch trình không xảy ra cascading rollback được gọi là lịch trình cascadeless.

Lịch trình Cascadeless

- Một lịch trình cascadeless là một lịch trình trong đó mỗi cặp giao dịch Ti và Tj:
  Nếu Ti đọc một hạng mục dữ liệu được ghi trước đó bởi Tj thì lệnh commit của Tj phải diễn ra trước lệnh đọc của Ti.
- Nghĩa là: hành động commit của giao dịch ghi phải diễn ra trước hành động đọc của giao dịch đọc.

TỔNG QUAN VỀ PHỤC HỒI

Nguyên tắc chung

- Khôi phục/phục hồi sau sự cố có nghĩa là CSDL khôi phục lại trạng thái nhất quán gần nhất trước thời điểm xảy ra sự cố.
- Để thực hiện được điều này, hệ thống cần lưu giữ các thông tin về sự thay đổi của dữ liệu bởi các giao dịch.

Cơ chế phục hồi cơ bản

- Nếu xảy ra sự cố nặng, hệ thống khôi phục lại dữ liệu được sao lưu trước đó và thực hiện lại (redo) các thao tác của các giao dịch đã được bàn giao.
- Nếu các sự cố nhẹ (1-4), hệ thống sẽ hủy bỏ các thao tác gây ra tính không nhất quán (undo). Trong một số tình huống, cần phải thực hiện lại một số thao tác (redo).

Phân loại cơ bản

- Về lý thuyết, ta có thể phân biệt 2 loại giải thuật phục hồi dựa trên:
  - Trì hoãn cập nhật (Deferred update): không cập nhật vật lý dữ liệu cho đến khi giao dịch hoàn tất.
  - Cập nhật ngay (Immediate update)
- Một giải thuật phục hồi thường gồm 2 phần:
  - Các hành động được thực hiện trong suốt quá trình hoạt động bình thường của hệ thống.
  - Các hành động thực hiện sau khi lỗi phát sinh.

Deferred Update Techniques

- Không cập nhật CSDL trên đĩa cho đến khi giao dịch được bàn giao.
- Trước đó, những thay đổi được lưu trong vùng làm việc cục bộ của giao dịch – local transaction workspace (buffers).
- Trong quá trình bàn giao, những thay đổi trước tiên được lưu trên log và sau đó ghi vào CSDL.

Deferred Update Techniques

- Nếu giao dịch lỗi trước khi bàn giao, nó sẽ không thay đổi CSDL do đó UNDO không cần thực hiện.
- Có thể cần thực hiện lại (REDO) một số tác động của giao dịch hoàn tất dựa trên log khi xảy ra sự cố khi chưa kịp ghi vào CSDL.

- ➔ **Giải thuật NO-UNDO/REDO.**

Immediate Update Techniques

- CSDL có thể được cập nhật bởi một số thao tác của giao dịch chưa được bàn giao.
- Việc cập nhật này thường được lưu vào log trên đĩa trước khi được cập nhật vào CSDL.
- Nếu sự cố xảy ra sau khi thay đổi CSDL nhưng trước thời điểm bàn giao, giao dịch phải được rollback.
- Thông thường cả UNDO và REDO đều cần.
- **➔ Giải thuật UNDO/REDO và biến thể UNDO/NO-REDO.**

Caching of Disk Blocks

- Cơ chế phục hồi liên quan chặt chẽ đến hệ điều hành cụ thể là việc đọc dữ liệu vào bộ nhớ (caching/buffering).
- Một hay nhiều disk page có chứa dữ liệu cần được thay đổi sẽ được đọc vào bộ nhớ chính và sau khi thay đổi sẽ được ghi ngược lại CSDL.
- Đây là công việc của hệ điều hành, tuy nhiên do quan trọng với quy trình phục hồi → được điều khiển bởi HQT CSDL.

Caching of Disk Blocks

- Thông thường 1 số vùng đệm của bộ nhớ được điều khiển bởi HQT CSDL (DBMS) gọi là DBMS cache.
- Một directory cho cache để quản lý disk page nào nằm ở vùng đệm nào (tương tự khái niệm page tables của HĐH)
  - <Disk page address, buffer location>

Caching of Disk Blocks

- Khi DBMS thực hiện 1 hành động lên dữ liệu X nào đó:
  - Đầu tiên kiểm tra cache directory xem disk page có chứa X có nằm trong cache không.
  - Nếu không thấy sẽ chép vùng nhớ cần thiết vào cache, có thể sẽ cần replace (flush) 1 số vùng trong cache.
    - Least recently used (LRU)
    - First-in-first-out (FIFO)
    - 1 số cơ chế riêng của mỗi DBMS
- Với mỗi vùng đệm kèm theo 1 bit gọi là dirty bit để đánh dấu vùng đệm đó có được cập nhật hay không.

Caching of Disk Blocks

- 2 cơ chế được thực thi khi ghi ngược từ buffer vào đĩa:
  - In-place updating: ghi buffer ngược lại vị trí cũ trên đĩa.
  - Shadowing: ghi buffer vào địa chỉ khác trên đĩa, do đó tồn tại nhiều phiên bản của dữ liệu → không cần thiết log tuy nhiên không khả thi trên thực tế
- Cách gọi thông thường:
  - Dữ liệu cũ trước khi cập nhật: before image – BFIM.
  - After image – AFIM.
- Trong kĩ thuật Shadowing, cả BFIM và AFIM đều được ghi trên đĩa.

KĨ THUẬT WRITE-AHEAD LOGGING

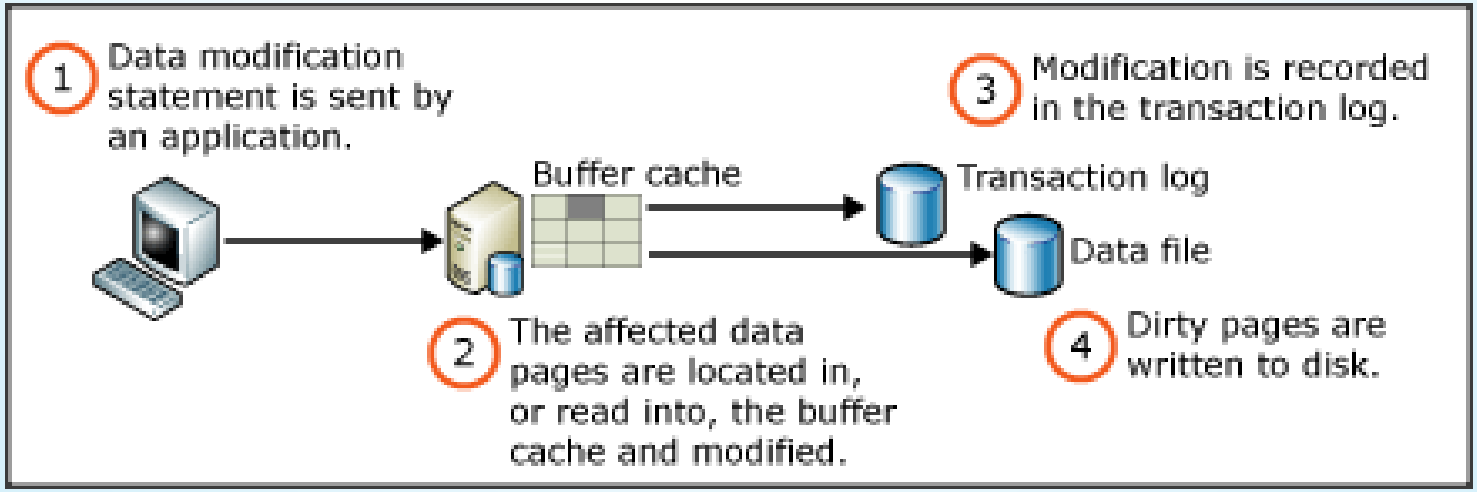
Khái niệm

- Trong quá trình ghi dữ liệu ngược từ cache vào đĩa, cần thiết phải sử dụng log để phục hồi trong những trường hợp xảy ra sự cố trong quá trình ghi này.
- Cơ chế phục hồi phải đảm bảo BFIM của dữ liệu được lưu trong log thích hợp và log được ghi vào đĩa trước khi BFIM được ghi đè bởi AFIM.

- 2 loại thông tin trong log cần thiết cho việc ghi dữ liệu:
  - Thông tin cần thiết cho việc UNDO: chứa giá trị cũ (BFIM) của dữ liệu, cần thiết cho việc phục hồi giá trị của dữ liệu về giá trị trước đó (bằng cách thay đổi giá trị trong database = giá trị BFIM).
  - Thông tin cần thiết cho việc REDO: giá trị mới (AFIM) của thao tác ghi, nó cần thiết cho việc thực hiện lại các thay đổi do thao tác ghi đó từ log (bằng cách thay đổi giá trị trong database = giá trị AFIM).

- DBMS cache lưu database disk blocks gồm:
  - Data blocks
  - Index blocks
  - Log blocks
- Khi một dòng trong log được cập nhật, nó được ghi vào log blocks hiện tại trong DBMS cache.
- Khi có sự cập nhật dữ liệu trong data blocks, records log tương ứng được ghi tiếp vào cuối log blocks trong DBMS cache.

- Trong cơ chế Write-Ahead Logging (WAL), log blocks tương ứng với data blocks (có sự thay đổi dữ liệu, cần được cập nhật lên đĩa) phải được ghi lên đĩa trước khi ghi chính data block đó ngược lại đĩa.



Thuật ngữ: Steal/No-steal

- No-steal: Nếu những cache page cập nhật bởi giao dịch không được phép ghi lên đĩa trước khi giao dịch commit. Thông thường dùng thêm 1 bit pin-unpin.
- Steal: Ngược lại, cho phép ghi cache ngay cả trước khi giao dịch commit.
  - Không cần buffer quá lớn để lưu các page có thay đổi.

Thuật ngữ: Force/No-force

- Force: Tất cả các page có cập nhật phải được ghi vào đĩa khi giao dịch commit. Ngược lại là No-force.
- Ưu điểm No-force:
  - Updated pages của những giao dịch đã commit có thể vẫn được giữ trong buffer để dùng cho các giao dịch khác do đó hạn chế chi phí I/O.
  - Rất lợi trong trường hợp một số vùng dữ liệu được thay đổi liên tục bởi nhiều giao dịch.

- Deferred Update: Không cập nhật CSDL trên đĩa cho đến khi giao dịch được bàn giao.
- → No-steal.

- Thông thường, DBMS thực hiện steal/no-force.

Giải thuật cho WAL

- BFIM của dữ liệu trên đĩa không được phép ghi đè cho đến khi tất cả các record log thuộc loại UNDO của các giao dịch cập nhật được ghi lên đĩa.
- Thao tác commit của một giao dịch không được phép hoàn tất cho đến khi tất cả log thuộc loại UNDO và REDO cho giao dịch đó được ghi lên đĩa.

- Để thuận lợi hơn cho quá trình phục hồi, hệ thống quản trị phục hồi có thể cần lưu giữ thông tin danh sách các giao dịch liên quan đang được xử lý:
  - Committed & aborted transactions.
  - Active transactions: các giao dịch đã bắt đầu nhưng chưa hoàn tất (commit).
- Quá trình phục hồi hiệu quả hơn.

Checkpoint

- Một loại dữ liệu khác trong log: [checkpoint]
- Thông thường được ghi vào log định kì tại thời điểm hệ thống ghi vào CSDL trên đĩa tất cả các page có chỉnh sửa trong buffer.
- Do đó tất cả giao dịch T đã được commit bởi dòng [commit, T] trong log trước thời điểm [checkpoint] không cần thiết REDO các thao tác write vì các thay đổi đã được ghi vào đĩa tại thời điểm [checkpoint].

- Hệ thống quản trị phục hồi quy định khoảng thời gian giữa 2 lần checkpoint.
- Có thể dựa trên thời gian, ví dụ mỗi m phút.
- Hoặc có thể dựa trên số giao dịch t được commit tính từ thời điểm checkpoint trước đó.
- Giá trị m hay t đó là các tham số hệ thống.

- Quy trình thực hiện một checkpoint:
  1. Tạm dừng hoạt động xử lý các giao dịch.
  2. Force-write tất cả buffer có thay đổi lên đĩa.
  3. Ghi [checkpoint] record vào log, force-write log lên đĩa.
  4. Tiếp tục lại các giao dịch đang thực hiện.

Fuzzy checkpoint

- Quá trình ghi buffer lên đĩa (do yêu cầu của bước 1) có thể làm chậm quá trình xử lý giao dịch.

- Để hạn chế điều này, hệ thống có thể tiếp tục xử lý giao dịch sau khi [checkpoint] record được ghi vào log mà không cần đợi bước 2 hoàn tất.

Fuzzy checkpoint

- Tuy nhiên trước khi bước 2 hoàn tất, [checkpoint] hợp lệ vẫn phải là [checkpoint] cũ.
- Để làm điều này, hệ thống duy trì một con trỏ chỉ đến [checkpoint] hợp lệ.
- Trước khi bước 2 hoàn tất, con trỏ vẫn chỉ đến [checkpoint] cũ trong log.
- Cho đến khi bước 2 hoàn tất, con trỏ được cập nhật đến [checkpoint] mới trong log.

- Nếu một giao dịch bị hủy bỏ sau khi đã có sự cập nhật dữ liệu, giao dịch đó cần được quay lui (rollback).
- Nếu có giá trị nào được thay đổi bởi giao dịch và ghi vào CSDL, nó cần được phục hồi lại giá trị trước đó (BFIM).
- Undo-type log entries được dùng cho việc phục hồi lại giá trị cũ của dữ liệu cần rollback.

- Nếu một giao dịch T bị rollback, tất cả giao dịch T’ đã đọc giá trị nào đó được ghi bởi T cũng phải được rollback…(Cascading rollback)
- Cascading rollback rất phức tạp và tốn thời gian do đó hầu hết cơ chế phục hồi được thiết kế đảm bảo việc cascading rollback không xảy ra.

(a)

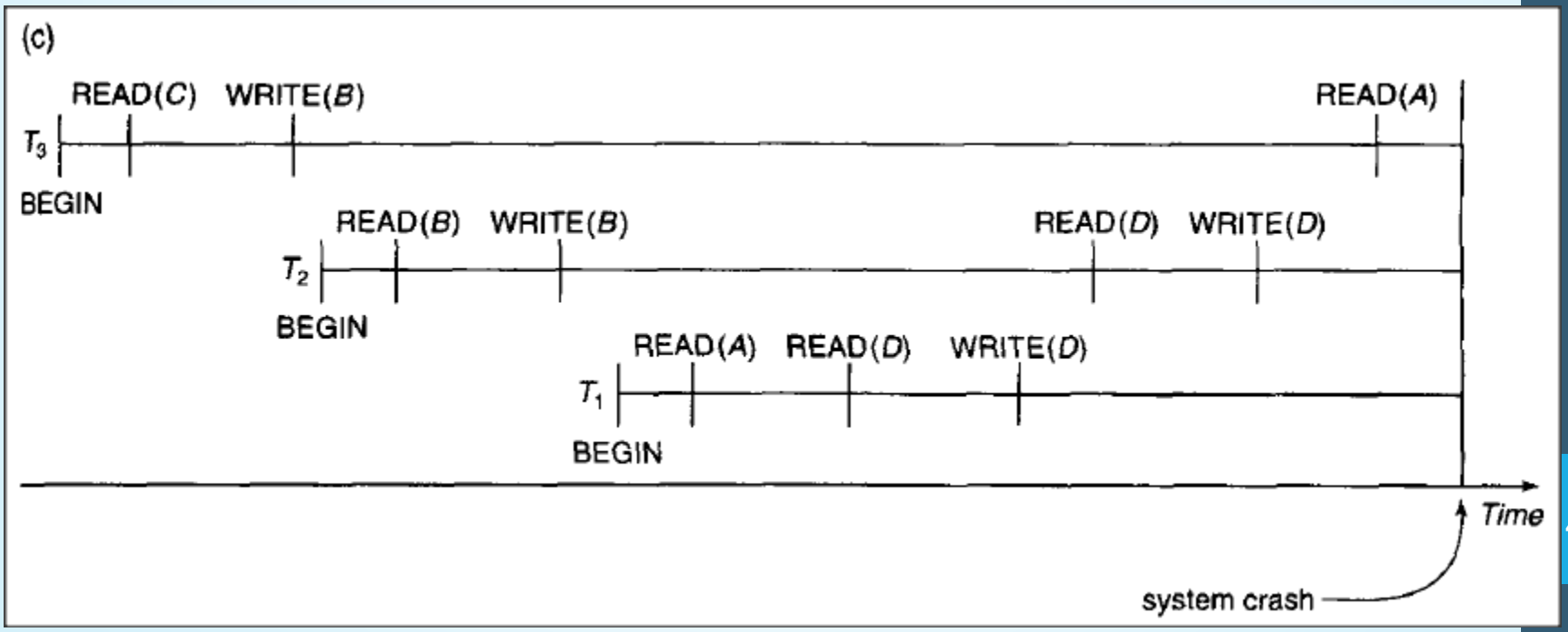
T_1	T_2	T_3
read_item(A)	read_item(B)	read_item(C)
read_item(D)	write_item(B)	write_item(B)
write_item(D)	read_item(D)	read_item(A)
	write_item(D)	write_item(A)

(b)

		A	B	C	D
		30	15	40	20
	[start_transaction, T_3]				
	[read_item, T_3, C]				
*	[write_item, T_3, B, 15, 12]		12		
	[start_transaction, T_2]				
	[read_item, T_2, B]				
**	[write_item, T_2, B, 12, 18]		18		
	[start_transaction, T_1]				
	[read_item, T_1, A]				
	[read_item, T_1, D]				
	[write_item, T_1, D, 20, 25]				25
	[read_item, T_2, D]				
**	[write_item, T_2, D, 25, 26]				26
	[read_item, T_3, A]				
	← system crash				

Ví dụ

- Chỉ các thao tác Write cần UNDO.
- Thao tác Read trong log chỉ cần cho việc xác định cascading rollback.



- Trên thực tế, cascading rollback không bao giờ xảy ra vì các phương pháp phục hồi thực tế luôn phải đảm bảo cascadeless hoặc strict schedule.
- Do đó, không cần thiết lưu các thao tác Read trong log.

KĨ THUẬT PHỤC HỒI DỰA TRÊN DEFERRED UPDATE

Khái niệm

- Ý tưởng chính của kĩ thuật Deferred Update là trì hoãn tất cả các cập nhật lên CSDL cho đến khi giao dịch hoàn tất (no-steal).
- Trong quá trình xử lý, các thay đổi được lưu trong log và trong cache.
- Sau khi giao dịch commit và log được force-write vào đĩa, các thay đổi mới được ghi vào CSDL trong đĩa.

Khái niệm

- Nếu một giao dịch bị lỗi trước khi commit, không cần phải UNDO bất cứ thao tác nào cả (do giao dịch không ảnh hưởng đến dữ liệu trên đĩa).

- Có thể sử dụng cho hệ thống có giao dịch ngắn và mỗi giao dịch thay đổi ít dữ liệu.

Khái niệm

- Mặc dù cách này làm đơn giản hóa qui trình phục hồi, tuy nhiên nó ít được sử dụng trong thực tế.
- Nguyên nhân: tiềm tàng khả năng hết bộ nhớ đệm do các thay đổi của giao dịch phải được lưu trong cache cho đến khi hoàn tất.

Typical deferred update protocol

- Một giao dịch không thể thay đổi CSDL trên đĩa cho đến khi nó hoàn tất.
- Một giao dịch không hoàn tất được trừ khi tất cả thay đổi được lưu vào log và log được force-write lên đĩa (Write Ahead Logging).
- Còn được gọi là Giải thuật phục hồi NO-UNDO/REDO.

Typical deferred update protocol

- REDO cần thiết trong trường hợp hệ thống lỗi sau khi giao dịch commit nhưng trước khi thay đổi được ghi lên đĩa hoàn tất.
- Phương pháp phục hồi liên quan chặt chẽ với phương pháp xử lý song hành:
  - Single-user environment: không có xử lý song hành.
  - Multi-user environment: thông thường, cấp độ xử lý song hành càng cao thì cơ chế phục hồi càng phức tạp, mất thời gian.

Giải thuật RDU_S

- Recovery using Deferred Update in a Single-user environment (RDU_S) algorithm.
- Sử dụng 2 danh sách giao dịch: giao dịch đã commit từ thời điểm checkpoint trước và giao dịch đang thực hiện (tối đa 1 giao dịch trong danh sách này).
- Thực hiện thao tác REDO cho tất cả thao tác Write của các giao dịch đã commit dựa trên log theo **thứ tự thực thi trong log**.
- Khởi động lại giao dịch đang thực hiện.

Thủ tục REDO

- Thủ tục REDO: Thực hiện lại các thao tác Write bằng cách đọc các dòng trong log [write, T, X, new_value] và gán giá trị X trong CSDL bằng giá trị new_value trong log (AFIM).
- Thao tác REDO phải có tính lũy đẳng – idempotent (việc thực hiện nhiều lần cũng như 1 lần).

Giải thuật RDU_S

- Trên thực tế, cả quy trình phục hồi cũng phải idempotent: Hệ thống lỗi trong khi đang phục hồi, lần phục hồi tiếp có thể REDO một số thao tác Write mà đã được REDO bởi lần phục hồi lỗi trước đó.
- → Kết quả phục hồi phải giống nhau.

Ghi chú

- Giao dịch duy nhất trong danh sách giao dịch đang thực hiện không ảnh hưởng đến CSDL (deferred update).
- Do đó nó được bỏ qua trong quá trình phục hồi.
- Tuy nhiên nó cần được khởi động lại tự động hoặc do người dùng.

Ví dụ

(a)

T_1	T_2
read_item(A)	read_item(B)
read_item(D)	write_item(B)
write_item(D)	read_item(D)
	write_item(D)

(b)

[start_transaction, T_1]
[write_item, T_1, D, 20]
[commit, T_1]
[start_transaction, T_2]
[write_item, T_2, B, 10]
[write_item, T_2, D, 25] ← system crash

The [write_item,...] operations of T_1 are redone.
T_2 log entries are ignored by the recovery process.

FIGURE 19.2 An example of recovery using deferred update in a single-user environment. (a) The READ and WRITE operations of two transactions. (b) The system log at the point of crash.

Giải thuật RDU_M

- Xét hệ thống sử dụng strict two-phase locking và [checkpoint] trong log.
- Sử dụng 2 danh sách giao dịch:
  - Các giao dịch đã commit từ thời điểm checkpoint trước.
  - Các giao dịch đang thực hiện.
- REDO tất cả thao tác Write của các giao dịch đã commit theo thứ tự được ghi trong log.
- Hủy bỏ các giao dịch đang thực hiện và khởi tạo lại.

Ví dụ

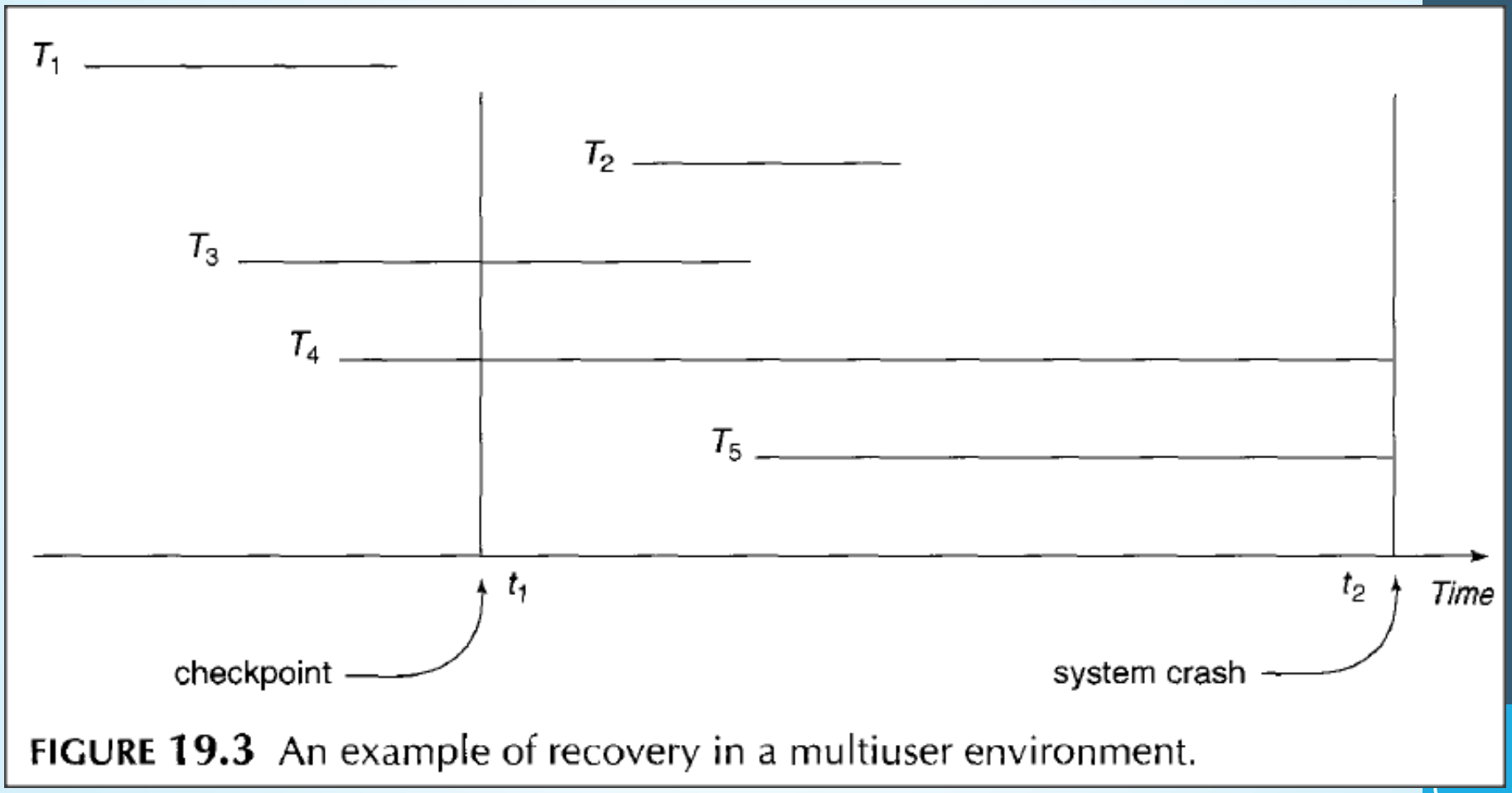


FIGURE 19.3 An example of recovery in a multiuser environment.

Ví dụ

- Không cần REDO T1.
- REDO các thao tác Write của T2 và T3.
- Hủy bỏ và gọi lại T4 và T5.

Cải tiến RDU

- Nếu dữ liệu X được cập nhật (thể hiện trong log) nhiều hơn 1 lần bởi các giao dịch đã commit, ta chỉ cần REDO giá trị cuối cùng của X trong quá trình phục hồi.
- Khi đó ta có thể bắt đầu từ cuối log, nếu dữ liệu được REDO hoàn tất thì thêm vào danh sách đã REDO.
- Trước khi thực hiện REDO, kiểm tra danh sách trước.

- Nếu 1 giao dịch bị hủy bỏ, chỉ cần thực hiện lại nó mà không ảnh hưởng gì đến CSDL.
- Hạn chế:
  - Giới hạn các giao dịch đồng thời vì khóa trên các hạng mục dữ liệu chỉ được mở khi giao dịch hoàn tất.
  - Cần không gian buffer lớn để lưu các dữ liệu có chỉnh sửa cho đến khi giao dịch được commit.
- Ưu điểm: giao dịch không bao giờ cần UNDO:
  - Không thay đổi dữ liệu cho đến khi giao dịch commit.
  - Không bao giờ đọc dữ liệu của giao dịch chưa commit bởi khóa.

(a)

T_1	T_2	T_3	T_4
read_item(A)	read_item(B)	read_item(A)	read_item(B)
read_item(D)	write_item(B)	write_item(A)	write_item(B)
write_item(D)	read_item(D)	read_item(C)	read_item(A)
	write_item(D)	write_item(C)	write_item(A)

(b)

[start_transaction, T_1]
[write_item, T_1, D, 20]
[commit, T_1]
[checkpoint]
[start_transaction, T_4]
[write_item, T_4, B, 15]
[write_item, T_4, A, 20]
[commit, T_4]
[start_transaction, T_2]
[write_item, T_2, B, 12]
[start_transaction, T_3]
[write_item, T_3, A, 30]
[write_item, T_2, D, 25] ← system crash

T_2 and T_3 are ignored because they did not reach their commit points.
T_4 is redone because its commit point is after the last system checkpoint.

Giao dịch không ảnh hưởng CSDL

- Một số giao dịch không ảnh hưởng đến CSDL như xuất message, báo cáo.
- Nếu giao dịch lỗi trước khi hoàn tất → report lỗi → cảnh báo user.
- Do đó các report này cũng phải được tạo sau khi các giao dịch hoàn tất.

KĨ THUẬT PHỤC HỒI DỰA TRÊN IMMEDIATE UPDATE

- Khi giao dịch thay đổi giá trị, CSDL có thể được cập nhật mà không cần đợi cho giao dịch đó hoàn tất.

- Việc cập nhật thay đổi lên đĩa trước hết vẫn cần được ghi lại trong log trên đĩa – cơ chế WAL để có thể phục hồi trong trường hợp xảy ra sự cố.

- Khi giao dịch thất bại, ta cần phải khôi phục (undo) tác động của các thao tác cập nhật ảnh hưởng đến CSDL của giao dịch đó.
- Thực hiện: rollback giao dịch, UNDO tác động của các thao tác Write.
- Về lý thuyết, chia làm 2 loại:
  - Tất cả cập nhật được ghi trên đĩa trước khi giao dịch commit: UNDO/NO-REDO.
  - Tổng quát: Cho phép giao dịch commit trước khi tất cả thay đổi được ghi vào đĩa: UNDO/REDO.

Giải thuật RIU_S

- Recovery using Immediate Update in a Single-user environment (RIU_S).
- Khi sự cố xảy ra, giao dịch đang thực hiện có thể đã cập nhật một số dữ liệu trên CSDL, những thay đổi đó cần phải được UNDO.
- Giải thuật RIU_S sử dụng thủ tục REDO ở phần trước và thủ tục UNDO.

Thủ tục UNDO

- Phục hồi (UNDO) một thao tác Write dựa trên việc xem xét các entry trong log [write, T, X, old_value, new_value] và đổi giá trị X trong CSDL thành old_value (BFIM).
- Việc phục hồi phải thực hiện theo thứ tự ***ngược lại*** với thứ tự của thao tác được ghi trong log.

Giải thuật RIU_S

- Sử dụng 2 danh sách giao dịch:
  - Danh sách các giao dịch đã được commit từ checkpoint trước.
  - Danh sách giao dịch đang thực hiện (max 1)
- UNDO tất cả thao tác Write của giao dịch đang thực hiện theo thủ tục UNDO.
- REDO các thao tác Write của các giao dịch đã commit dựa trên log theo thứ tự được ghi trong log theo thủ tục REDO.

Giải thuật RIU_M

- Xử lý song hành: Sử dụng strict 2PL → xảy ra deadlock → hủy bỏ và UNDO.
- Sử dụng 2 danh sách giao dịch:
  - Danh sách các giao dịch đã được commit từ checkpoint trước.
  - Danh sách các giao dịch đang thực hiện.
- UNDO tất cả thao tác Write của các giao dịch đang thực hiện theo thủ tục UNDO theo thứ tự ngược với thứ tự ghi trong log.
- REDO các thao tác Write của các giao dịch đã commit dựa trên log theo thứ tự được ghi trong log theo thủ tục REDO.

KĨ THUẬT PHỤC HỒI DỰA TRÊN SHADOW PAGING

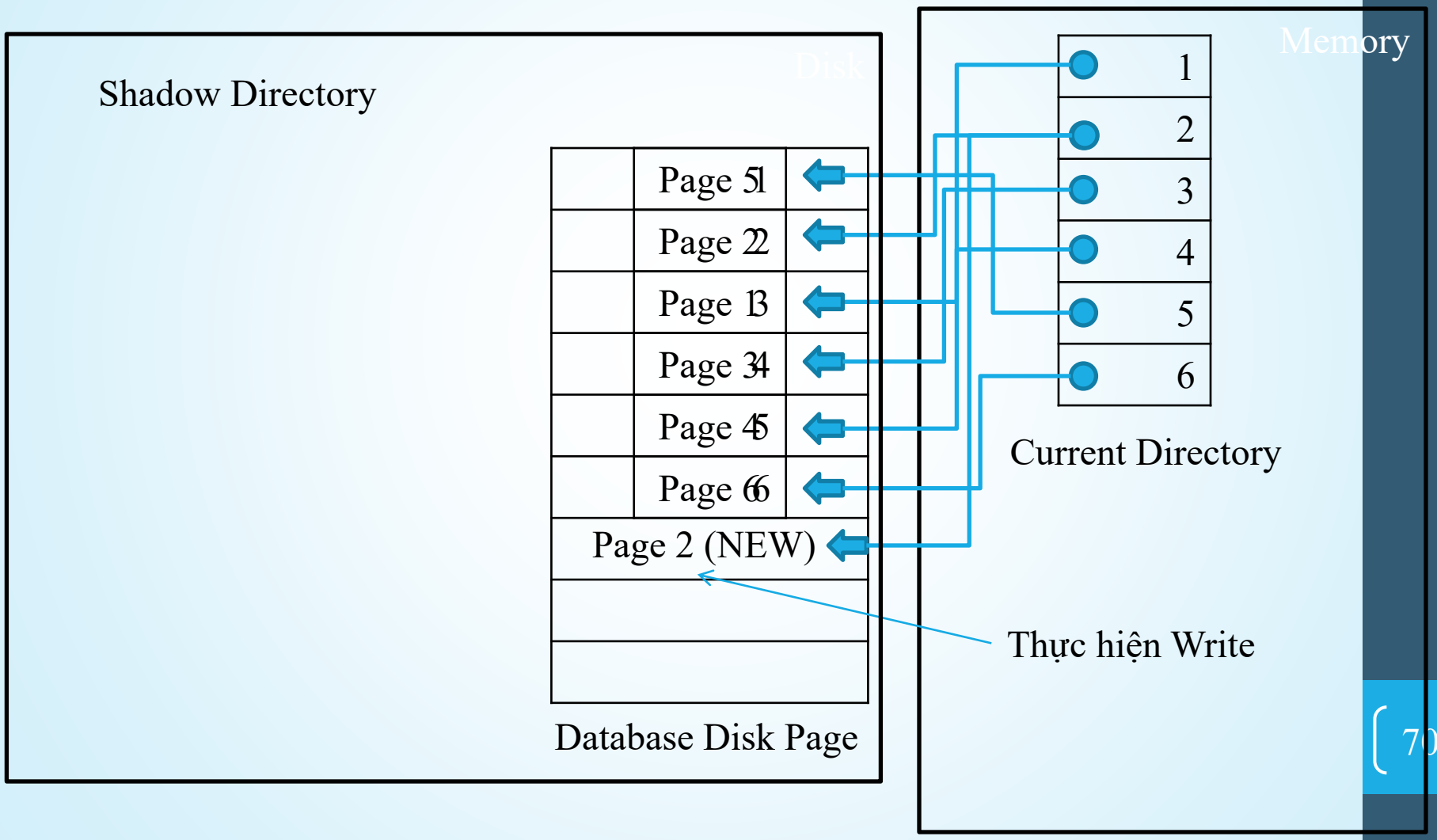
Kĩ thuật Shadow Paging

- Không cần đến log trong Single-user environment, trong multi-user environment, cần đến log để xử lý song hành.
- Xem CSDL được tạo thành bởi tập hợp disk page kích thước cố định.
- Dùng Directory lưu địa chỉ các page của Database trên đĩa và được quản lý trong bộ nhớ chính nếu nó không quá lớn.
- Khi bắt đầu 1 giao dịch, directory hiện tại được lưu lại 1 bản (shadow directory) và lưu vào đĩa.

Kĩ thuật Shadow Paging

- Trong suốt quá trình thực hiện các giao dịch, chỉ xử lý trên current directory.
- Khi thực hiện write, một page mới được tạo chứ không ghi đè lên page cũ.
- Địa chỉ tương ứng của page trong current directory được thay đổi chỉ đến page mới được tạo.

Minh họa Shadow Page



Phục hồi trong Shadow Paging

- Phục hồi đơn giản là thực hiện:
  - Xóa các page được chỉnh sửa trong disk (Page 2 New).
  - Hủy bỏ current directory.
- Trạng thái dữ liệu trước khi xảy ra sự cố được tái lập thông qua shadow directory.
- Khi commit giao dịch: hủy bỏ các shadow directory.
- Không cần undo hay redo dữ liệu ➔ **NO-UNDO/NO-REDO**

Phục hồi trong Shadow Paging

- Với multi-user environment, log và checkpoint được sử dụng.
- Hạn chế:
  - Khó kiểm soát vị trí các page trong đĩa gần nhau.
  - Vấn đề khi ghi directory xuống đĩa trong trường hợp directory quá lớn.
  - Vấn đề về garbage collection khi giao dịch commit.
  - Phải đảm bảo tính nguyên tử trong quá trình xử lý current và shadow directory.

Một số vấn đề khác

- Giải thuật phục hồi ARIES.
- Vấn đề phục hồi trong hệ thống multiDB.
- Sao lưu và phục hồi trong các sự cố lớn.

1. Backup – Recovery trong SQL Server

- SQL Server cung cấp các cơ chế recovery và backup như sau:
  - Recovery model: Simple, Full, Bulk-logged.
  - Backup: Full backup, Transaction Log backup, Differential backup, File/File Group backup.

- Các kiểu sao lưu (Backup)

  Full Backup (Sao lưu toàn bộ)

  Transaction Log Backup (Sao lưu nhật ký giao tác)

  Differential Backup (Sao lưu vi sai)

1. Backup – Recovery trong SQL Server

- **Full Backup (Sao lưu toàn bộ)**
  - Sao lưu toàn bộ dữ liệu, bao gồm cả stored procedure, view, hàm do người dùng định nghĩa, và transaction log.
  - File tạo ra có phần mở rộng .bak.
- Nên thực hiện Full Backup khi:
  - CSDL có kích thước nhỏ.
  - CSDL ít có các thay đổi.
- Cú pháp lệnh:

BACKUP DATABASE {*database_name* | @database_name_var}
TO <*backup_device*> [, ...n]
[WITH
[FORMAT]
[[,] {INIT | NOINIT}]]
]

1. Backup – Recovery trong SQL Server

- **Transaction Log Backup (Sao lưu nhật ký giao tác)**
  - Sao lưu tập tin ghi sổ (transaction log): các lệnh làm thay đổi dữ liệu.
  - Cần thực hiện Full Backup đầu tiên, sau đó mới sử dụng các lần Transaction Log Backup tiếp theo.
- Nên sử dụng khi:
  - CSDL thường xuyên bị thay đổi.
  - Thời gian thực hiện Full Backup quá lâu.
- Cú pháp lệnh:

```
BACKUP LOG {database_name | @database_name_var}
TO <backup_device> [, ...n]
[WITH
[{INIT | NOINIT}]
]
```

1. Backup – Recovery trong SQL Server

- **Differential Backup (Sao lưu vi sai)**
  - Sao lưu tất cả các dữ liệu đã thay đổi kể từ lần Full Backup gần nhất.
- Nên sử dụng Differential Backup nếu:
  - Dữ liệu thường xuyên bị thay đổi.
  - Muốn rút ngắn thời gian thực hiện của các lần backup.
- Cú pháp lệnh:

```
BACKUP DATABASE {database_name | @database_name_var}
TO <backup_device> [, ...n]
[WITH
        [DIFFERENTIAL]
]
```

- Cú pháp sao lưu theo file hoặc filegroup cụ thể:
  - BACKUP DATABASE {database_name | @database_name_var} [<file_or_filegroup> [, ...m]] TO <backup_device> [, ...n]]
    <file_or_filegroup>: { FILE = {logical_file_name | @logical_file_name_var} | FILEGROUP = {logical_filegroup_name | @logical_filegroup_name_var} }

1. Backup – Recovery trong SQL Server

- **Differential Backup (Sao lưu vi sai)**
- Cú pháp sao lưu theo file hoặc filegroup cụ thể:

BACKUP DATABASE {*database_name* | @database_name_var}
[<*file_or_filegroup*> [, ...m]] **TO** <backup_device> [, ...n]]

<*file_or_filegroup*>:
{
FILE = {*logical_file_name* | @logical_file_name_var}
| FILEGROUP = {*logical_filegroup_name* |
    @logical_filegroup_name_var}
}

1. Backup – Recovery trong SQL Server

- Thực hiện Backup trong SQL Server Management Studio
  - Bước 1: Nhấn chuột phải vào tên CSDL cần sao lưu → chọn Tasks → chọn Back Up...
  - Bước 2: Trong hộp thoại Back Up Database, chọn kiểu sao lưu (Backup type): Full hoặc Differential.
  - Bước 3: Chỉ định đường dẫn và tên file lưu bản sao. Lưu ý phải nhập đầy đủ phần mở rộng .bak hoặc .trn. Có thể dùng nút Remove để huỷ vị trí lưu file cũ, và nút Add để chỉ định vị trí lưu file mới.
  - Bước 4: Ở mục Options, thiết lập chế độ ghi đè dữ liệu (Overwrite media): "Append to the existing backup set" cho phép lưu nhiều bản sao CSDL từ các lần sao lưu khác nhau vào cùng một file; "Overwrite all existing backup sets" chỉ giữ lại một bản sao duy nhất của lần sao lưu cuối cùng.

1. Backup – Recovery trong SQL Server

- **Các mô hình Recovery (Recovery Model)**

Mô hình	Giải thích
Simple	Chỉ sử dụng được Full backup và Differential backup. Tập tin ghi sổ (transaction log) sẽ tự động bị "truncate" nếu thiếu không gian lưu trữ.
Full	Sử dụng được tất cả các loại backup (Full, Differential, Transaction Log, File/Filegroup).
Bulk-logged	Tương tự mô hình Full, nhưng chỉ ghi lại một số lệnh cập nhật nhất định (không ghi lại toàn bộ).

1. Backup – Recovery trong SQL Server

- **Phục hồi CSDL (Restore)**
- Các bước thực hiện phục hồi CSDL bằng giao diện đồ hoạ trong SQL Server Management Studio:
  - Bước 1: Nhấn chuột phải vào tên CSDL → chọn Tasks → Restore → chọn Database... (hoặc Files and Filegroups..., Transaction Log... tuỳ nhu cầu).
  - Bước 2: Chọn radio button "From device" để chỉ định file CSDL cần phục hồi trên ổ đĩa, sau đó nhấn nút Add để chỉ định vị trí chứa file backup.
  - Bước 3: Trong danh sách các backup set, chọn phiên bản bản sao lưu CSDL cần dùng để khôi phục.
  - Bước 4: Ở mục Options, chọn Recovery state phù hợp. Chọn "RESTORE WITH NORECOVERY" (để CSDL ở trạng thái chưa hoạt động, không rollback các transaction chưa commit) khi cần phục hồi tiếp từ một bản Full backup và nhiều bản Differential backup nối tiếp nhau.

1. Backup – Recovery trong SQL Server

Một số gợi ý khi sao lưu / phục hồi CSDL

- Càng thực hiện sao lưu nhiều lần thì càng giảm được rủi ro khi có sự cố xảy ra.
- Với các CSDL quan trọng, có nhiều thay đổi trong ngày, nên thực hiện nhiều lần sao lưu trong một ngày.
- Full Backup là phương pháp an toàn nhất, nhưng nếu thực hiện Full Backup quá nhiều lần sẽ tốn nhiều dung lượng bộ nhớ.
- Do đó, nên áp dụng chiến lược kết hợp như sau:
  - Thực hiện Full Backup một lần vào lúc bắt đầu ngày làm việc (trước khi bắt đầu làm việc với CSDL).
  - Thực hiện nhiều lần Differential Backup trong ngày (định kỳ, ví dụ khoảng vài tiếng một lần).
  - Thực hiện nhiều lần Transaction Log Backup trong ngày.
- Khi có sự cố xảy ra, tiến hành phục hồi theo trình tự sau:
  - Phục hồi CSDL sử dụng bản sao lưu Full Backup.
  - Phục hồi CSDL sử dụng bản sao lưu Differential Backup gần nhất và ở trước thời điểm xảy ra sự cố.
  - Phục hồi CSDL sử dụng bản sao lưu Transaction Log Backup gần nhất và ở trước thời điểm xảy ra sự cố.

2. Đặt lịch sao lưu tự động (Maintenance Plan)

2. Đặt lịch sao lưu tự động (Maintenance Plan)

- Đặt lịch sao lưu tự động là việc tạo một kế hoạch (Maintenance Plan) cho SQL Server, để SQL Server tự động thực hiện sao lưu CSDL tại các thời điểm được ấn định sẵn, không cần thao tác thủ công.
- Điều kiện: để tạo lịch sao lưu tự động, người dùng phải đăng nhập bằng Login ID có vai trò "sysadmin".

2. Đặt lịch sao lưu tự động (Maintenance Plan)

- **Các bước tạo Maintenance Plan**
  - Bước 1: Trong Object Explorer, mở mục Management → nhấn chuột phải vào Maintenance Plans → chọn New Maintenance Plan.
  - Bước 2: Đặt tên cho Maintenance Plan (ví dụ: "database bkup plan").
  - Bước 3: Trên trang cấu hình, nhấn vào biểu tượng lịch ở góc phải subplan để ấn định thời gian thực hiện sao lưu tự động. Trong hộp thoại Job Schedule Properties: chọn Occurs = Daily, chọn giờ thực hiện (Daily frequency → Occurs once at) — nên đặt vào thời điểm CSDL ít được sử dụng nhất (ví dụ 4:00 AM).
  - Bước 4: Từ khung Toolbox (Maintenance Plan Tasks), kéo thả tác vụ Back Up Database Task sang khung thiết kế bên phải.
  - Bước 5: Nhấn đúp chuột vào Back Up Database Task vừa thêm để cấu hình: chọn Backup type là "Full"; chọn các CSDL cần sao lưu trong mục Database(s); chọn đường dẫn lưu bản sao (Folder); nhập "bak" vào mục Backup file extension.
  - Bước 6: Vì việc sao lưu được thực hiện thường xuyên có thể dẫn đến đầy bộ nhớ server, cần kéo thả thêm tác vụ Maintenance Cleanup Task từ Toolbox sang khung thiết kế, nối tiếp sau Back Up Database Task.
  - Bước 7: Nhấn đúp chuột vào Maintenance Cleanup Task để cấu hình: chỉ định vị trí lưu file bản sao CSDL và phần mở rộng của bản sao (Folder, File extension); chỉ định khoảng thời gian giữ lại các bản sao (ví dụ: giữ lại các bản sao được sao lưu trong vòng 4 tuần và tự động xoá các bản sao cũ hơn).

2. Đặt lịch sao lưu tự động (Maintenance Plan)

- **Lưu ý quan trọng**
- SQL Server chỉ thực hiện sao lưu tự động thành công khi dịch vụ SQL Server Agent đang chạy (running). Nếu SQL Server Agent bị tắt (stopped), có thể khởi động lại thông qua:
  - SQL Server Management Studio (nhấn chuột phải vào SQL Server Agent → Start).
  - SQL Server Configuration Manager.

3. System Stored Procedure

3. System Stored Procedure

- System Stored Procedure là những Stored Procedure được cung cấp sẵn khi cài đặt SQL Server. Chúng rất hữu ích khi thực hiện các hành động quản trị và xem thông tin về các đối tượng trong SQL Server.
- Danh sách đầy đủ các System Stored Procedure có thể tham khảo tại địa chỉ: http://msdn.microsoft.com/en-us/library/ms187961.aspx. Trên trang này, các System Stored Procedure được phân theo nhóm, trong đó có hai nhóm quan trọng và thường được sử dụng nhiều nhất:
  - Database Engine Stored Procedures: chứa nhiều System SP dùng để xem thông tin các đối tượng trong SQL Server.
  - Security Stored Procedures: chứa các System SP dùng cho mục đích quản trị bảo mật hệ thống.

3. System Stored Procedure

- **Một số System Stored Procedure hữu ích**

Thủ tục	Mô tả
sp_help [<tên đối tượng CSDL>]	Trả về thông tin đối tượng CSDL (bảng, view, stored procedure...). Nếu không truyền tham số, trả về tổng hợp tất cả đối tượng trong CSDL hiện thời.
sp_HelpText <tên>	Trả về văn bản (định nghĩa) của stored procedure, hàm người dùng định nghĩa, trigger, hay view không mã hoá.
sp_HelpDb [<tên CSDL>]	Trả về thông tin của CSDL được chỉ định; nếu không chỉ định tham số, trả về thông tin của toàn bộ các CSDL.
sp_Helpfile	Xem tên vật lý và các thuộc tính của các tập tin liên quan đến CSDL hiện tại. Dùng để xác định tên vật lý của database cần detach hoặc attach.

3. System Stored Procedure

- **Một số System Stored Procedure hữu ích**

Thủ tục	Mô tả
sp_Who [<ID đăng nhập>]	Trả về thông tin người đang đăng nhập và các tiến trình đang chạy. Nếu không chỉ định tham số, trả về thông tin của toàn bộ người dùng đang hoạt động.
sp_Columns <tên>	Trả về thông tin các cột được định nghĩa trong bảng hoặc view được chỉ định.
sp_tables	Xem danh sách các đối tượng có thể truy vấn trong CSDL hiện tại (tất cả các đối tượng có thể xuất hiện trong mệnh đề FROM).
sp_columns <tên>	Xem thông tin các cột trong một table hoặc view.

3. System Stored Procedure

- **Một số System Stored Procedure hữu ích**

Thủ tục	Mô tả
sp_depends <tên>	Xem danh sách các Stored Procedure, View phụ thuộc vào (tham chiếu đến) bảng hoặc view trong CSDL.
sp_helpsrvrole ['<Tên Server Role>']	Trả về danh sách tất cả Server Role nếu không truyền tham số; hoặc trả về thông tin mô tả cho Server Role được chỉ định.
sp_password '<Password cũ>', '<password mới>', '<Login ID>'	Thay đổi mật khẩu (password) cho một Login ID.
sp_helplogins	Cung cấp thông tin về Login ID cùng với thông tin các Database User liên kết với Login ID này.
sp_addlogin	Thêm một Login ID mới.
sp_adduser	Thêm một Database User cho một Login ID.

3. System Stored Procedure

- **Thiết lập phím tắt cho các Stored Procedure hệ thống**
  - SQL Server Management Studio cho phép thiết lập phím tắt (keyboard shortcut) để gọi nhanh các System Stored Procedure thường dùng, thông qua menu Tools → Options → Environment → Keyboard.
  - Bảng phím tắt mặc định (Standard keyboard scheme):

Phím tắt	Stored Procedure
Alt + F1	sp_help
Ctrl + F1	sp_helptext
Ctrl + 1	sp_who
Ctrl + 2	sp_lock
Ctrl + 3	sp_HelpDb

IV. Bài tập

Bài tập 1

- Cho ba giao dịch T1, T2, T3 cùng thao tác trên các hạng mục dữ liệu X, Y với lịch trình xen kẽ (interleaved) S như sau (thứ tự thực thi từ trái sang phải):

S: r1(X); r2(Y); w1(X); r3(X); w2(Y); c2; r1(Y); w3(X); c3; w1(Y); c1

a) Lịch trình S có khả phục hồi (recoverable) hay không? Chứng minh bằng cách chỉ ra các mối quan hệ đọc/ghi giữa các giao dịch.

b) Lịch trình S có phải là cascadeless không? Giải thích.

c) Nếu S không thỏa một trong hai tính chất trên, hãy đề xuất cách điều chỉnh thời điểm commit (giữ nguyên thứ tự các thao tác đọc/ghi) để lịch trình trở thành cascadeless.

Bài tập 2

- Cho lịch trình S2 giữa ba giao dịch T1, T2, T3 thao tác trên hạng mục dữ liệu A:

S2: r1(A); w1(A); r2(A); w2(A); r3(A); abort(T1)

a) Xác định các giao dịch bị ảnh hưởng dây chuyền khi T1 abort. Giải thích chi tiết chuỗi phụ thuộc.

b) Vì sao hiện tượng này được gọi là cascading rollback? Nêu hệ quả của nó đối với hiệu năng hệ thống.

c) Đề xuất một điều kiện trên lịch trình để hiện tượng này không bao giờ xảy ra, và cho biết trong thực tế các DBMS thường đảm bảo điều kiện đó bằng cơ chế nào.

Bài tập 3

- Hệ thống áp dụng kỹ thuật phục hồi Deferred Update (NO-UNDO/REDO) trong môi trường đa người dùng (RDU_M), sử dụng Strict 2PL. Log ghi nhận theo thứ tự sau (hệ thống bị system crash ngay sau dòng cuối cùng):

a) Áp dụng giải thuật RDU_M, xác định danh sách giao dịch cần REDO và danh sách giao dịch cần huỷ bỏ/khởi động lại (restart).

b) Trình bày cụ thể thao tác REDO cần thực hiện (giao dịch nào, hạng mục nào, giá trị mới là gì).

c) Giải thích vì sao T3 và T4 không cần phải UNDO dù đã có thao tác Write ghi trong log.

STT	Bản ghi log	Ghi chú
1	[start_transaction, T1]	
2	[write_item, T1, A, 10, 20]	A: 10→20
3	[commit, T1]	
4	[checkpoint]	
5	[start_transaction, T2]	
6	[write_item, T2, B, 5, 15]	B: 5→15
7	[start_transaction, T3]	
8	[write_item, T3, C, 100, 200]	C: 100→200
9	[commit, T2]	
10	[start_transaction, T4]	
11	[write_item, T4, D, 1, 2]	D: 1→2
12	[write_item, T3, E, 50, 60]	E: 50→60 ← system crash

END

Tham khảo chương 23

Fundamentals of Database System, 6th Edition